MODULE 07 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

ReplicaSets and Deployments

Keeping the right number of Pods, and changing them without downtime.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 7 of 35

admin@sthithapragna.com

A ReplicaSet keeps a number of Pods running. A Deployment manages change.

What a ReplicaSet is made of

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata: {name: web}
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
      - {name: web, image: quayside/web:5.2.0}
```

5 How many Pods to keep running.

7 Which Pods count: any Pod whose labels match.

10 The template's labels must match the selector, or the API rejects it.

The selector gotcha

- **It counts any matching Pod,** including ones it did not create.
- **Create a Pod by hand** with the same labels, and the ReplicaSet may delete one to stay at its number.
- **A Deployment's selector is immutable** once created. Changing it means a new Deployment.
- **ReplicationController** is the legacy predecessor. You will still see it named.

> ON THE EXAM
>
> Labels are the glue. Unique, deliberate labels keep controllers from adopting Pods they should not.

One Deployment, several ReplicaSets

Summarised from kubectl get replicasets and kubectl rollout history

REPLICASET	REVISION	IMAGE	PODS
booking-api-7d9f6c5b8d	3	quayside/booking-api:2.4.1	3
booking-api-5b8c4d7f9a	2	quayside/booking-api:2.4.0	0
booking-api-66f7a1c2e4	1	quayside/booking-api:2.3.9	0

ON THE EXAM

Each template version gets its own ReplicaSet, named with a pod-template-hash. Old ones are kept at zero for rollback.

ReplicaSet versus Deployment

REPLICASET

Keeps a count

- Maintains N matching Pods
- No rollout logic
- No history or rollback
- Usually created by a Deployment

DEPLOYMENT

Manages versions

- Owns and manages ReplicaSets
- Rolling updates or Recreate
- Revision history and rollback
- Pause, resume and restart

THE TIEBREAKER

For a stateless app, create a Deployment. The Deployment manages ReplicaSets, which manage Pods.

What triggers a rollout

- **Only changes to the Pod template,** spec.template: image, environment, resources, template labels.
- **Scaling is not a rollout.** Changing replicas creates no new ReplicaSet and no new revision.
- **kubectl rollout restart** forces a fresh rollout by stamping a timestamp into the template.
- Changing a ConfigMap the Pods read does **not** trigger a rollout on its own.

> ON THE EXAM
>
> New template, new ReplicaSet, new revision. Anything else changes the current ReplicaSet in place.

A rolling update, step by step

4 replicas, maxSurge 1, maxUnavailable 0

01
Surge
New ReplicaSet scales to 1. Five Pods exist.

02
Ready
The new Pod passes its readiness check.

03
Retire
Old ReplicaSet scales from 4 to 3.

04
Repeat
New 2, old 2; then new 3, old 1.

05
Done
New 4, old 0, kept for rollback.

ON THE EXAM

Capacity never drops below four, because no old Pod leaves until a new one is Ready.

maxSurge and maxUnavailable

SETTING	DEFAULT	MEANS	WITH 4 REPLICAS
`maxSurge`	25%, rounded up	Extra Pods allowed above replicas	Up to 5 Pods
`maxUnavailable`	25%, rounded down	Pods allowed to be unavailable	At least 3 available

ON THE EXAM

Both cannot be zero. maxUnavailable 0 means no capacity loss; a bigger maxSurge means a faster rollout.

RollingUpdate versus Recreate

ROLLINGUPDATE (DEFAULT)

Gradual, no downtime

- Old and new run side by side briefly
- Paced by maxSurge and maxUnavailable
- Needs versions that can coexist
- Stalls safely if new Pods never get Ready

RECREATE

All out, then all in

- Every old Pod is stopped first
- Then all new Pods start
- A short outage on every deploy
- For versions that must never overlap

THE TIEBREAKER

Default to RollingUpdate. Choose Recreate only when two versions running at once would break something.

Readiness is what paces a rollout

- **A new Pod counts as available** only once it is Ready, and stays Ready for minReadySeconds.
- **A broken release stalls,** it does not spread: old Pods keep serving.
- **progressDeadlineSeconds**, 600 by default, marks it as failed with ProgressDeadlineExceeded.
- **There is no automatic rollback.** Someone, or some tool, must undo it.

> ON THE EXAM
>
> A stuck rollout is safe but not self-fixing. Roll back, or fix forward.

Driving a rollout from the command line

```
$ kubectl set image deployment/booking-api api=quayside/booking-api:2.5.0
$ kubectl rollout status deployment/booking-api
Waiting for rollout to finish: 2 of 4 updated replicas are available...
$ kubectl rollout history deployment/booking-api
REVISION  CHANGE-CAUSE
3         image 2.4.1
4         image 2.5.0
$ kubectl rollout undo deployment/booking-api
$ kubectl rollout undo deployment/booking-api --to-revision=2
```

1. Changes the template's image, so a rollout starts.
2. Watches progress until it completes, or fails.
4. Lists revisions. CHANGE-CAUSE comes from an annotation.
8-9. Back one revision, or to a specific one.

The rollout family, and its neighbours

COMMAND	DOES
`kubectl rollout status`	Waits for a rollout to finish, or reports it stuck
`kubectl rollout history`	Lists revisions; --revision=N shows one in detail
`kubectl rollout undo`	Returns to the previous, or a named, revision
`kubectl rollout pause / resume`	Batches several template changes into one rollout
`kubectl rollout restart`	Replaces every Pod without changing the spec
`kubectl scale --replicas=N`	Changes the count. Not a rollout

> ON THE EXAM
>
> status, history, undo and restart also work on DaemonSets and StatefulSets. Pause and resume are for Deployments only.

What a rollback really does

- **Undo copies an old template** back into the Deployment. The old ReplicaSet scales back up.
- **It becomes a new revision.** Revision 3 rolled back becomes revision 5; 3 leaves the list.
- **With GitOps, roll back in Git.** Otherwise the GitOps tool re-applies the broken version.
- Record why with the kubernetes.io/change-cause annotation.

> ON THE EXAM
>
> A rollback is just another rollout, to an older template.

> Quayside's booking API Deployment has 4 replicas and the default RollingUpdate settings. A new image is rolled out.
>
> The capacity planner asks for the limits that hold at every moment during the rollout.

What are the limits during this rollout?

A. At most 4 Pods exist, and at least 4 are available

B. At most 6 Pods exist, and at least 2 are available

C. At most 5 Pods exist, and at least 3 are available

D. At most 8 Pods exist, and none need be available

◆ PAUSE AND DECIDE

What are the limits during this rollout?

A At most 4 Pods exist, and at least 4 are available
That needs both settings at zero, which is not allowed.

B At most 6 Pods exist, and at least 2 are available
That would be 50% surge and 50% unavailable.

C At most 5 Pods exist, and at least 3 are available
25% of 4 is 1: surge of 1, unavailable of 1.

D At most 8 Pods exist, and none need be available
Describes nothing real; Recreate would allow zero, but never eight.

THE SIGNAL

Defaults: 25% surge rounded up, 25% unavailable rounded down. Do the arithmetic on the replica count.

> Quayside's payment worker holds an exclusive session with the card processor. Two versions running at once, even briefly, cause duplicate charges.
>
> A short outage during each release is acceptable to the business.

Which Deployment strategy fits?

A RollingUpdate with maxSurge 1 and maxUnavailable 0

B RollingUpdate with the default 25% settings

C A canary, sending a small share to the new version first

D Recreate, so all old Pods stop before new ones start

◆ PAUSE AND DECIDE

Which Deployment strategy fits?

A RollingUpdate with maxSurge 1 and maxUnavailable 0

Guarantees overlap: old and new run together by design.

B RollingUpdate with the default 25% settings

Still runs both versions side by side during the rollout.

C A canary, sending a small share to the new version first

A canary is overlap on purpose, the opposite of what is needed.

D Recreate, so all old Pods stop before new ones start

No overlap at all, at the cost of the outage they accepted.

THE SIGNAL

'Must never overlap' plus 'outage acceptable' is Recreate.

> An engineer sets a new image on the web Deployment, which has 3 replicas and default settings. The image tag has a typo.
>
> kubectl rollout status hangs. One new Pod shows ImagePullBackOff. The three original Pods are still Running.

What is the situation, and the best next step?

A Old Pods still serve; run kubectl rollout undo

B It will roll back automatically at the deadline

C Delete the old ReplicaSet to force the change

D Scale to zero and back up to restart the rollout

◆ PAUSE AND DECIDE

What is the situation, and the best next step?

A Old Pods still serve; run kubectl rollout undo

The rollout stalled safely. Undo returns to the working template.

B It will roll back automatically at the deadline

The deadline only reports failure. Nothing rolls back by itself.

C Delete the old ReplicaSet to force the change

Deletes the only working Pods: a self-inflicted outage.

D Scale to zero and back up to restart the rollout

Takes the site down, and the new template is still broken.

THE SIGNAL

A stalled rollout protects you. Undo it, or fix the tag and roll forward.

> Yesterday Quayside scaled the booking API from 3 to 6 replicas. Today they changed one environment variable in the Pod template.
>
> kubectl rollout history shows only one new revision since last week.

Why is there only one new revision?

A revisionHistoryLimit is set to 1

B Scaling never changes the Pod template

C Revisions are recorded only with --record

D The scale revision is stored on the Pods

◆ PAUSE AND DECIDE

Why is there only one new revision?

A revisionHistoryLimit is set to 1

Possible in theory, but nothing suggests it; the default is 10.

B Scaling never changes the Pod template

Only template changes create revisions. The env change made one.

C Revisions are recorded only with --record

--record only filled in the change cause, and it is deprecated.

D The scale revision is stored on the Pods

There is no such thing as a scale revision.

THE SIGNAL

Revisions track the Pod template. Replicas are not part of it.

Module 7 in five lines

- **ReplicaSet keeps a count**; a **Deployment manages ReplicaSets** and change.
- **Only template changes** create a new ReplicaSet and revision. Scaling does not.
- Defaults: **maxSurge 25% up, maxUnavailable 25% down**. Readiness paces the rollout.
- **RollingUpdate** for zero downtime; **Recreate** when versions must not overlap.
- **No automatic rollback.** Use rollout status, history and undo.

Deployments suit stateless apps. Next: **choosing a workload** for everything else.